

SERUM

Content-Aware Agentic Containment of Malware Under an Unobserved Payload

Semantic Epidemic Response under Unknown Malware

Amritha S.

A defensive research testbed · cybersecurity · network science · agentic AI

PART 1

The Problem

Why containing malware is hard when you can't see the attack

Malware spreads like an epidemic -- but a picky one

- A self-propagating worm moves host to host, like a disease over a contact network.
- The malware twist: a worm exploiting vulnerability *c* can infect a neighbour ONLY IF that neighbour runs the vulnerable software (carries *c*).
- So a machine without the weakness is simply immune to THIS worm -- the germ bounces off.
- Consequence: the worm travels only the SUB-network of machines that share its target weakness, not the whole network.

The picky germ, concretely

Think of each machine as 'wearing socks' -- the software it runs. The worm only infects one colour.

RED

Machines running the vulnerable software (carry the target CVE) -- CAN catch it.

BLUE

Machines running something else -- the worm bounces off. Totally safe.

SPREAD

The worm walks the network but only crosses into RED machines.

So the 'real' network the worm uses = the RED sub-network, not the whole graph.

The physical network is NOT the propagation graph

The network a worm actually traverses is a payload-specific SUB-graph.

- Physical topology: every machine and every link.
- Propagation graph: only the machines carrying the target CVE, and the links between them.
- These can look completely different -- the vulnerable machines might be scattered in a corner, not spread across the hubs.
- A defender who confuses the two will defend the wrong thing.

Why the usual defense wastes its budget

- You never have enough budget to protect every machine -- you pick a few.
- Standard heuristics (degree, betweenness immunization) pick the network hubs.
- But a hub is irrelevant to THIS worm if it can't run the exploit -- a blue-socks machine. Every band-aid spent on it is wasted.
- On real networks the vulnerable machines are often NOT the hubs -- so 'protect the hubs' barely helps.

The twist that makes it a real research problem

The defender NEVER observes the payload -- only who is infected.

- You don't get told the worm's target vulnerability -- no signature, no payload capture.
- You observe: the set of currently-infected machines, and your own asset inventory.
- So you must DEFEND the vulnerable sub-network without being told which sub-network it is.
- This elevates it from 'SI on a subgraph' to a genuine inference problem.

The clue hiding in plain sight

Because spread is vulnerability-gated, every infected machine MUST carry the target CVE.

- A machine can only be infected by spread if it was vulnerable to the worm.
- So each newly-infected machine is a hard CONSTRAINT: the true exploit lies in its software profile.
- Intersect those profiles across all infected machines -> the set of consistent exploits shrinks.
- SERUM turns this into online Bayesian inference -- a POMDP whose hidden state is the exploit -- and acts under that belief.

PART 2

Thesis & Contributions

What we claim, and what we honestly do not

Thesis

- A defender that reasons about WHAT is spreading -- the payload's target vulnerability -- contains a worm far more efficiently, with far less collateral disruption, than one that sees only network STRUCTURE.
- ...and this holds even when the payload is never directly observed -- it is inferred from the shape of the outbreak.

Honest from the start: what is and isn't new

- NOT novel (we claim none of this):
- vulnerability-gated spread = multitype bond percolation; structure-only immunization baselines are classical; POMDP defense & group testing are established.
- Our contribution -- the COUPLING no prior work occupies:
- online inference of the unobserved exploit from a vulnerability-gated cascade, with a checkable identifiability condition on OBSERVABLE attributes, driving budgeted content-aware containment, on REAL NVD/CVE data. (The cell CyGym 2025 leaves open.)

The four contributions (we lead with the finding)

- C1 -- Content-aware containment: on a REAL topology it substantially beats every structure-only baseline (-28.4%, $p=1.7e-7$).
- C2 -- An identifiability CHARACTERIZATION: identifiable iff infected profiles intersect in a singleton; + a measured group-testing sample-complexity rate.
- C3 -- A REAL-DATA evaluation (NVD/CVE) with paired statistics and honest scope.
- Robustness note -- attacking the inference backfires; poisoning is absorbed (graceful degradation, not a superiority claim).

PART 3

Problem Formulation

The containment POMDP, precisely

The SERUM POMDP -- the pieces

- Hosts & profiles: graph $G=(V,E)$; each host v carries $X(v)$, the CVEs it is exploitable by.
- Hidden state: the worm's target c^* . The defender never sees it.
- Vulnerable subgraph: carriers(c^*) and their links -- the graph the worm can use.
- Observation each step: the infected set (not c^*) + the static inventory.
- Actions (budget B/step): PATCH (immunize, keeps host online), ISOLATE (remove, costs availability), SEGMENT (cut a link).

How a single spread step works

Infected u tries to infect susceptible neighbour w . Success is GATED:

CHECK

Does w carry the target CVE c^* ? (Is w red-socks?)

NO

w is immune to this worm. Nothing happens -- ever.

YES

w gets infected with probability β (transmissibility).

Repeat over all infected->susceptible edges -> the outbreak grows within the red subgraph.

The objective -- three things at once

Containment is multi-objective -- not just infection count.

- Minimize final/peak INFECTION (how much of the fleet falls).
- Minimize AVAILABILITY loss (isolating machines takes them offline).
- Minimize TIME-TO-CONTAINMENT (stop it fast).
- This is why PATCH vs ISOLATE matters: patching immunizes without downtime, but you only dare patch precisely once confident which machines are truly at risk.

PART 4

Theory -- Identifiability

When can you actually figure out the exploit?

The question the theory answers

Given only who's infected, when is the hidden exploit uniquely recoverable?

- If you can pin the exact CVE, you can defend its subgraph exactly.
- If two CVEs are indistinguishable from the infections seen, you must hedge across both.
- We want a CHECKABLE condition -- one a defender can evaluate in advance from its inventory.
- It turns out to be a simple set-containment condition on observable attributes.

Worked example: narrowing to the exploit

supp = intersection of infected profiles. Watch the candidate set shrink as machines fall.

Host A

infected. Runs {CVE-1, CVE-2, CVE-5}. Candidates = {1,2,5}.

Host B

infected. Runs {CVE-2, CVE-5, CVE-9}. $\{1,2,5\} \cap \{2,5,9\} = \{2,5\}$.

Host C

infected. Runs {CVE-5, CVE-7}. $\{2,5\} \cap \{5,7\} = \{5\}$.

Intersection collapsed to a single CVE -> the exploit is identified: it must be CVE-5.

The identifiability theorem

c^* is identifiable from a saturating outbreak over R iff $\text{intersection}(\text{profiles in } R) = \{c^*\}$.

- 'Saturating' = the outbreak has infected everything reachable in the vulnerable subgraph.
- Single-CVE intersection -> you know the attack exactly.
- Two or more -> those CVEs are 'confusable'; watching this outbreak can't separate them.
- HONEST: the belief's support EQUALS this intersection by construction -- so it's a characterization of the observation model, not a deep theorem. The '100% validation' is a consistency check on the code, not a prediction.

Confusability = the subset order

One CVE hides behind another exactly when its victims are a subset of the other's.

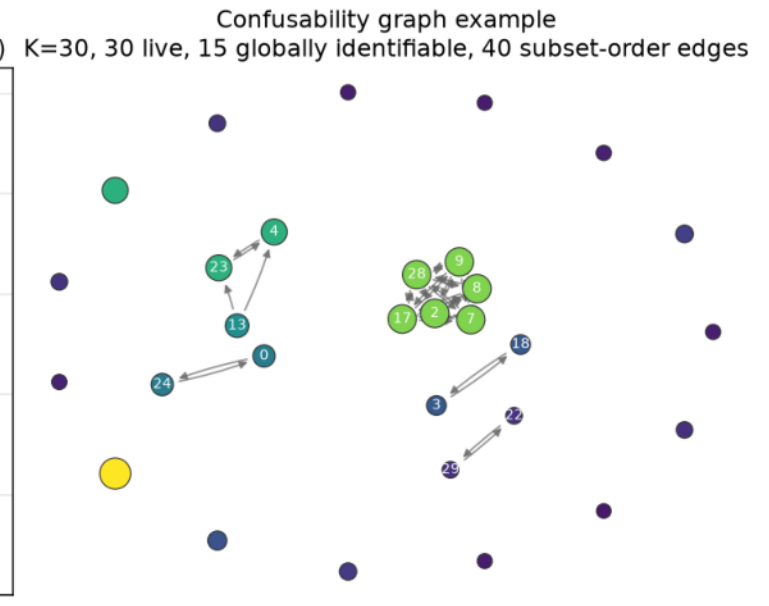
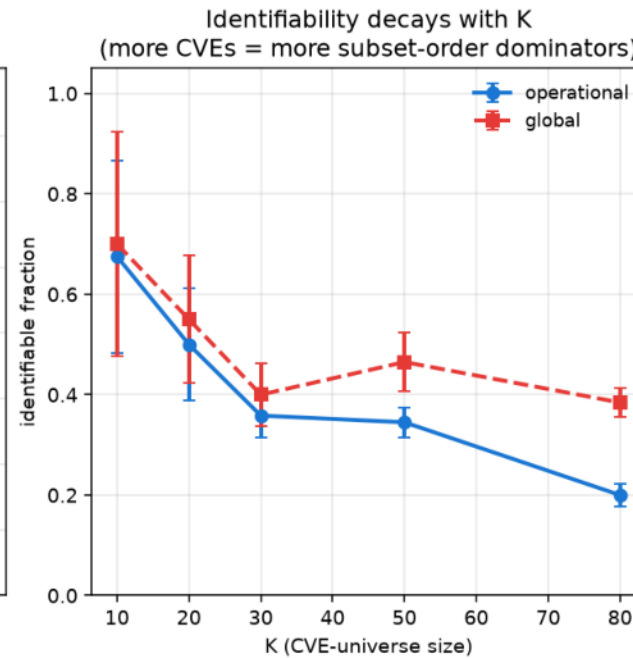
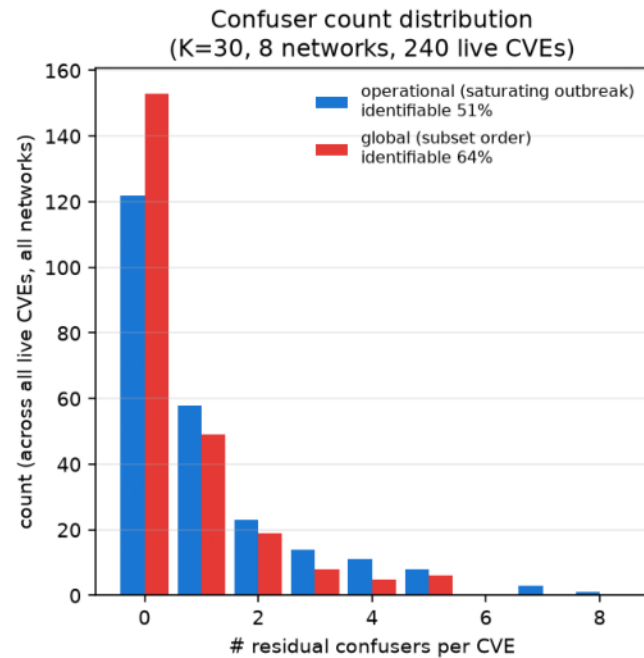
- If carriers(c) is a SUBSET of carriers(c'), every c -victim also carries c' -- so a c -outbreak can never rule c' out. c' is a confuser of c .
- A CVE is globally identifiable iff no other CVE's carrier set is a superset of its own.
- Powerful for DEFENSE: even if you can't tell c from c' , defending c' 's (larger) subgraph OVER-COVERS the true victims -- you still protect them.

It's group testing -- realized by contagion

The whole inference is online, graph-induced, adversarial group testing.

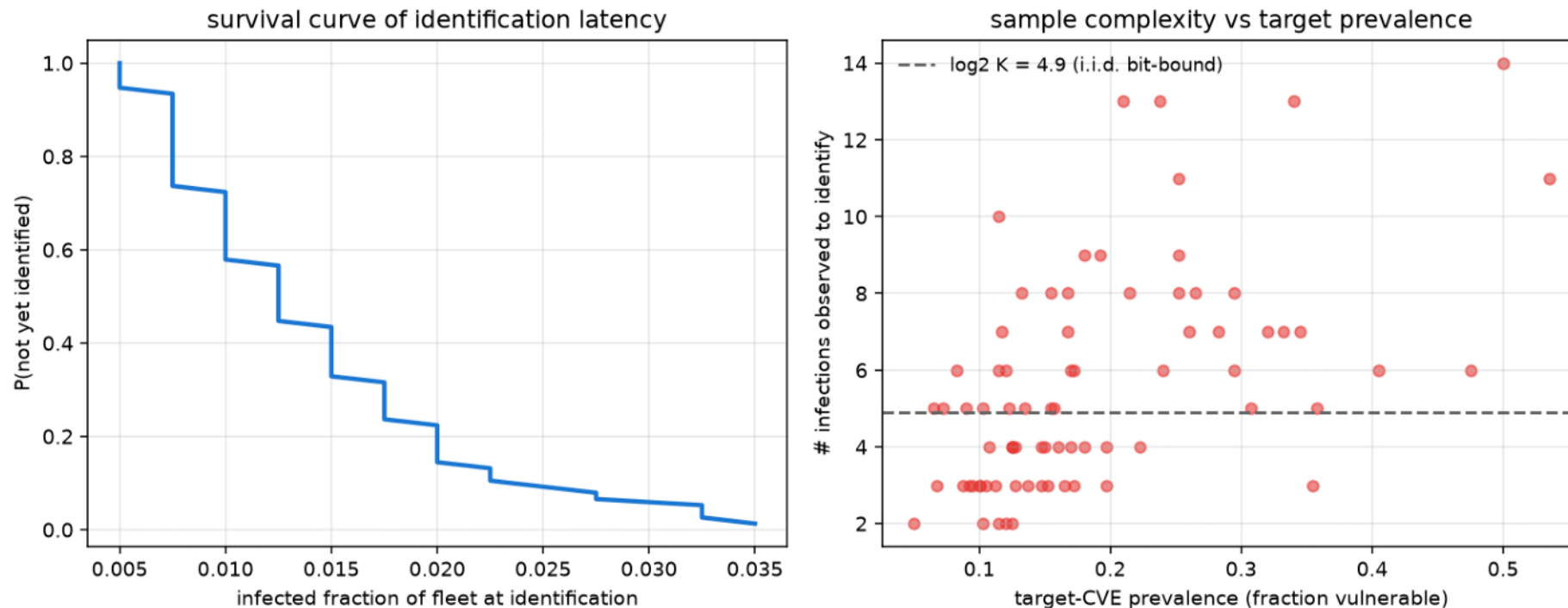
- Classic group testing: identify a defective by pooling and testing subsets.
- Here each infection is one 'positive test' whose profile-set is intersected into the running candidate set (Renyi separating systems; Kautz-Singleton cover-free families).
- Our twist: the 'tests' aren't designed -- they're the fleet's given software profiles -- and outcomes are produced by an adversarial, graph-constrained spread, observed online.
- So identification inherits the group-testing $\log_2(K)$ bit-bound.

Confusability structure (real NVD networks)



How to read it: left: most CVEs have ZERO confusers (identifiable). middle: identifiable fraction decays as the CVE universe K grows. right: a drawn slice of the confusability graph.

Sample complexity -- how fast you identify the exploit



How to read it: distribution of infections until the candidate set collapses to one CVE. Median ~ 5 infections $\sim 1.02 \times \log_2(K)$ -- essentially the information-theoretic minimum. Real profile correlation bends the rate TOWARD the optimum.

PART 5

The Method

The belief, and the content-aware policy

The belief update, step by step

The agent keeps a probability over which CVE is the attack, and sharpens it each step.

PRIOR	Start from CVE prevalence (or a CVSS/LLM threat-intel prior for cold start).
OBSERVE	New infections arrive. Each one must carry the true CVE.
UPDATE	Down-weight (soft) or eliminate (hard) CVEs inconsistent with new victims.
ACT	Take the expectation of the defense score over the current belief.

Early: belief broad -> hedge. Later: belief sharp -> commit. It acts under uncertainty.

Soft vs hard likelihood -- why soft wins

Real detection is noisy -- hard elimination is brittle, soft is robust.

- HARD consistency eliminates any CVE absent from an infected host's profile -- exact, but a single false-positive detection can wrongly eliminate the TRUE CVE.
- SOFT likelihood DOWN-WEIGHTS instead -- one bad observation can't exclude the truth.
- Soft is robust to noisy detection and to deliberate belief-poisoning, and is what we deploy in every deployment-facing experiment.

The content-aware policy -- the scoring rule

Score each frontier host by its belief-weighted EXPOSED-VULNERABLE DEGREE.

- For a host v : how many neighbours are (a) still susceptible AND (b) exploitable by a CVE the belief thinks is live? Average that over the belief = v 's expected onward infections.
- Defend the highest-scoring hosts. It generalizes 'degree' immunization to the uncertain-payload regime, reducing to plain degree when everything is vulnerable.
- While uncertain -> ISOLATE (blunt but certain). Once the belief's support collapses -> PATCH (immunize precisely, keep hosts online).

PART 6

Results

Synthetic, real-topology, and real-data -- with honest scope

Experimental design -- why it's trustworthy

- Paired design: same graph, target CVE, seeds, and infection coin-flips for every defender in a trial -- slashes variance, makes deltas real.
- Randomized target CVE per trial -- no cherry-picked attack.
- Real vulnerabilities from NVD/CVE; per-CVE transmissibility from CVSS.
- Paired Wilcoxon + Holm-Bonferroni correction on headline claims; we report per-trial WIN RATES, not just means.

Headline -- synthetic networks, real CVEs

Defender	Infected	Availability	Contain@step
No defense	36.6%	100%	14.2
Best structural (degree/betweenness)	3.4%	95.5%	4.5
SERUM -- content-aware (ours)	1.8%	98.5%	3.0
Oracle (is told the exploit)	1.1%	100%	2.0

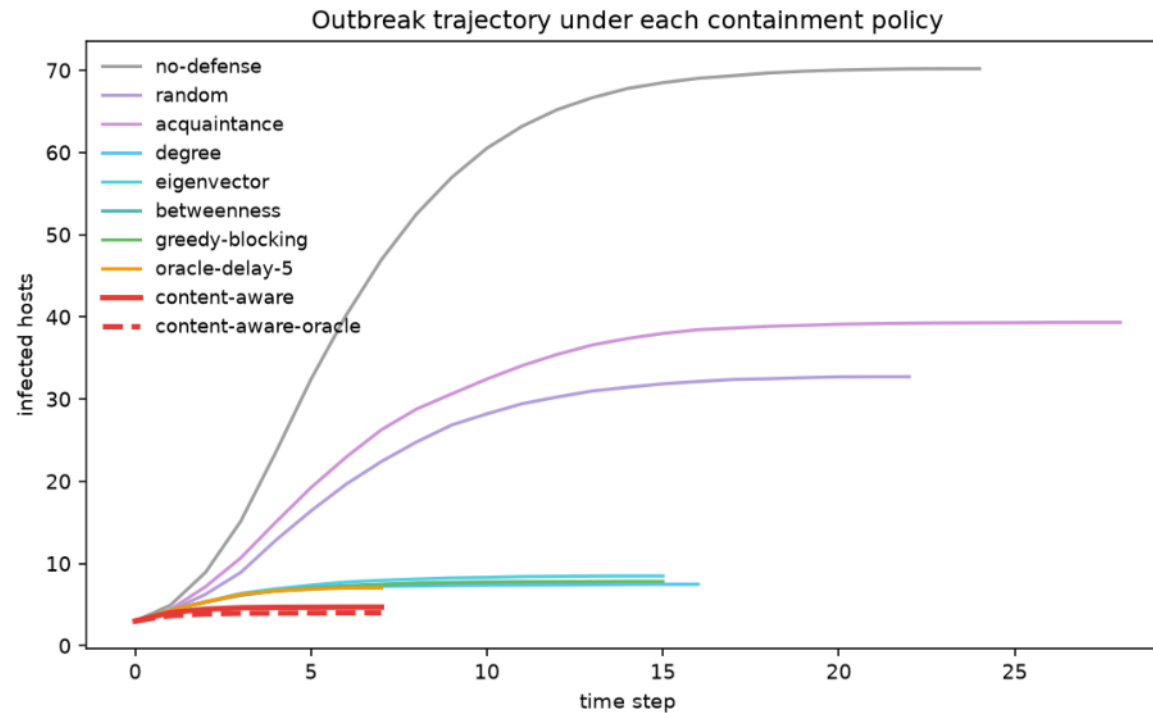
What it means: Beats the best structural baseline on infection AND availability AND speed -- without being told the attack. Honest: the absolute gap here is small; the decisive gap is on real topology.

Flagship -- a REAL organisational network

Defender	Infected	Availability
No defense	20.1%	100%
Degree immunization	18.8%	92.2%
Betweenness (best structural)	17.6%	92.5%
SERUM -- content-aware (ours)	11.7%	97.5%
Content-aware oracle (bound)	9.1%	100%

What it means: SNAP email-Eu-core: 1004 REAL hosts, 42 REAL departments as software zones, real NVD CVEs. Structure-only BARELY helps (17.6% vs 20.1%) -- vulnerable departments aren't the hubs. Content-aware cuts to 11.7%: -28.4%, $p=1.7e-7$. The result we lead with.

Flagship -- infection over time (real topology)

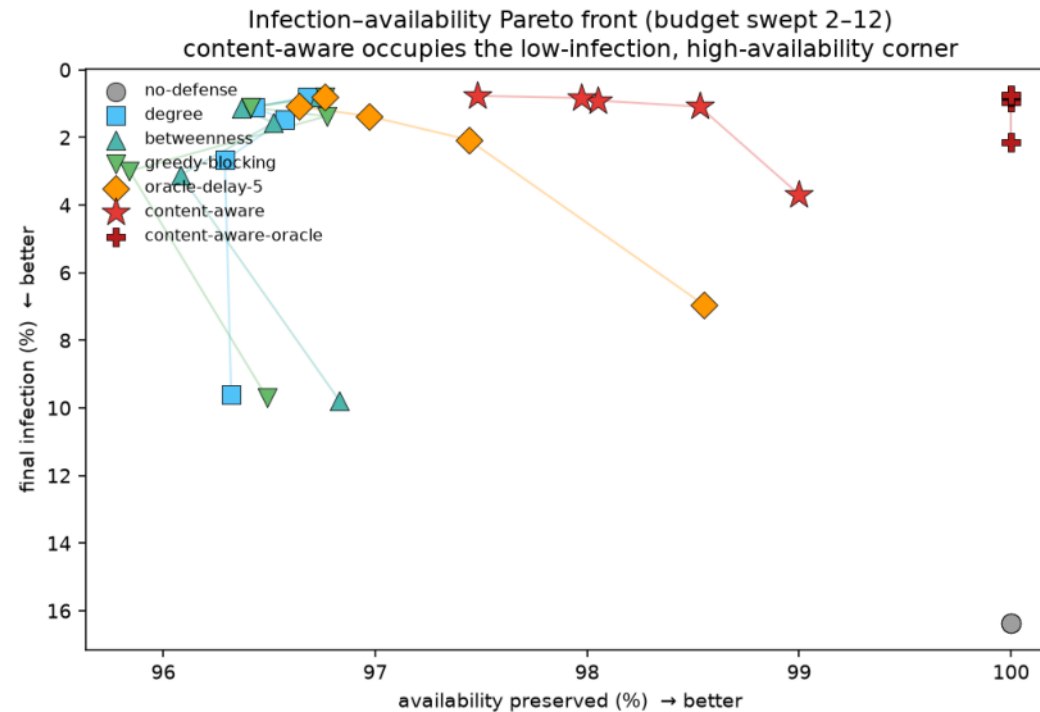


How to read it: structure-only curves sit on top of no-defense; the content-aware curve is far below, hugging the oracle. The gap between them is the value of content-awareness.

The lead result is NOT a fragile minority effect

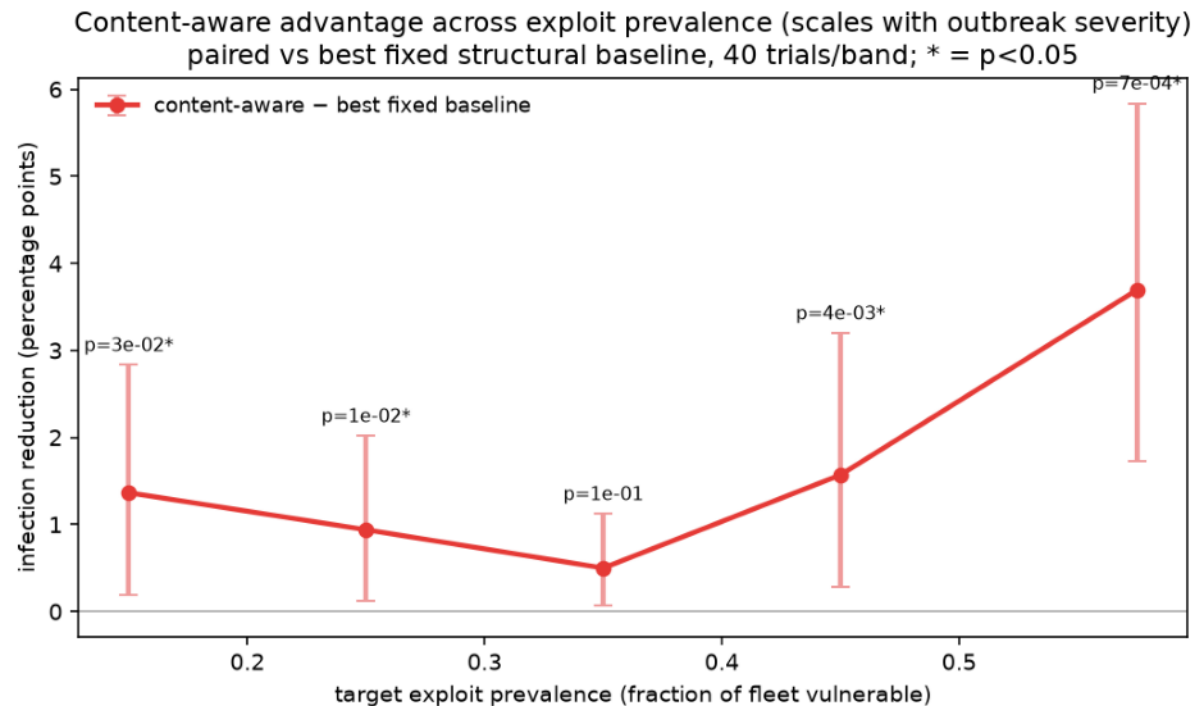
- 20 / 20 paired outbreaks WON on the real topology -- vs betweenness AND the ensemble oracle ($p=8.8e-5$). 37/40 in a budget-8 replication.
- On the small synthetic margins, content-aware wins only a MINORITY of trials -- a fair critic would pounce. On the REAL topology it wins essentially EVERY outbreak, by 5.9pp absolute.
- Honest boundary: on hub-aligned or near-universal-exploit regimes the edge is small or vanishes. We say so.

Pareto dominance -- infection vs availability



How to read it: each point = one defender at one budget; down-and-right is better. Content-aware sits in the bottom-right corner at EVERY budget -- lower-infection AND higher-availability than every structural baseline.

The advantage scales with outbreak severity



How to read it: relative reduction vs the best structural baseline across prevalence bands -- it grows with severity.
Honest: in low-prevalence bands content-aware wins a MINORITY of trials and one band isn't significant.

Head-to-head vs the CLOSEST prior systems

Defender (of this class)	Infected	vs ours
DAVA-style -- data-aware, exploit-blind	1.70%	+43.8% · p=2.8e-4
CyGym-style -- static prior, no update	1.15%	+16.6% · p=1.1e-2
SERUM -- content-aware (ours)	0.95%	--

What it means: We beat BOTH nearest systems. DAVA even underperforms plain degree -- vaccinating exposed-but-non-exploitable hosts wastes budget, i.e. the thesis. Honest: these are faithful reimplementations of each system's STANCE, not the full originals.

When does the **ONLINE** inference actually matter?

Content-AWARENESS is the driver; the online UPDATE is a smaller refinement.

- Under a GOOD prior, online updating beats a frozen prior by only +0.19pp -- consistent with our ablation that freezing the belief costs almost nothing.
- It earns its keep when the prior is WRONG: under a misleading prior the edge roughly DOUBLES to +0.44pp ($p=1.8e-2$).
- So online inference is a refinement that matters most under bad threat intel -- NOT the main source of the win. We say this plainly.

PART 7

Robustness

Can an attacker defeat the defender by attacking its inference?

Attacking the inference BACKFIRES

An attacker who picks a hard-to-identify payload does not help itself.

- A confusable exploit's victims are a SUBSET of its confuser's victims.
- So even a hedged belief that can't tell them apart still DEFENDS the true victims -- they're inside the set it protects.
- Empirically the content-aware edge is LARGER under an evasive attacker (+22.4%) than a random one (+17.7%).
- Inference-evasion is not a winning strategy -- a structural guarantee, not luck.

Belief poisoning, and the honest limit

A stronger attacker plants FAKE infections to mislead the belief.

- Fix: a RobustAgent audits its belief vs the real spread and hedges to structure when they diverge -- poisoning is one-shot, the real worm keeps revealing the truth.
- Against a white-box, audit-AWARE adaptive poisoner it holds up to 6% of the fleet; only at an extreme 10% does the attacker gain a small edge (which fails multiple-comparison correction).
- HONEST: this is GRACEFUL DEGRADATION, not a win. Under poisoning the content-aware advantage evaporates and the agent falls back to -- never below -- structure. A safety net.

Two more robustness stresses

- Imperfect detection sensors: model missed detections + persistent false alarms. Content-aware degrades GRACEFULLY (positive edge at 9 of 10 noise points); false alarms are the sensitive channel. Not a new significant win.
- Is the win a manufactured artifact? We sweep the monoculture 'homophily' knob 0->0.8. The edge is significant even at 0 (no monoculture, +0.26pp, $p=6.5e-4$). So it's NOT a knob artifact -- homophily only controls spatial clustering. (This rebuts the knob worry, not the semi-synthetic-data limitation.)

PART 8

Breadth & Honesty

Generalization, positioning, and the limits we own

Breadth -- application & learned policy

- IoT botnet (Mirai-style): device-firmware monoculture; host value = uplink Mbps, so blast radius = DDoS capacity conscripted. Content-aware cuts DDoS blast -8.71pp vs degree (20/20 wins).
- Learned policy: a cross-entropy-method policy over belief features MATCHES the hand-designed agent and beats degree/betweenness -- and independently learns to weight the belief features most, validating the design (learning UNDER exploit uncertainty, not vanilla GNN-RL).

The three nearest systems -- cite, then differentiate

Prior work	Shares with SERUM	The gap SERUM fills
CyGym (2025)	vuln-gated spread + cost model	STATIC prior; no online update
SCENARIOID (KDD'23)	infer mechanism from cascade	OFFLINE; no ID guarantee
Hoffmann (ICML'20)	identifiability from cascades	LATENT edges (harder setting)

What it means: We cite all three prominently and state each delta in the same breath -- including, honestly, that our observable-attribute setting is in some ways EASIER than Hoffmann's, not strictly stronger.

Honest limitations (several load-bearing)

We state these plainly -- a paper is stronger for owning them.

- L1 -- Not yet validated on real HOST-LEVEL data (measured per-host inventories are proprietary). The single most important gap.
- L2 -- Online inference is a REFINEMENT, not the driver. L3 -- host<->CVE mapping is MODELED.
- L4 detection noise, L5 adaptive adversary, L6 multiplicity -- addressed (with caveats).
- L7 -- Abstract model: no packet-level malware, scanning, or C2 layer.

L1 -- ready to close in ONE command

We can't close L1 without proprietary data -- but we made it ready for the instant it exists.

- We will NOT fabricate a real inventory, and another synthetic model wouldn't satisfy L1.
- We ship a TESTED pipeline (serum/data/real_inventory.py, scripts/validate_real_inventory.py) that builds the network from a real scan (measured host,CVE findings) + a topology edge list, taking each host's vulnerabilities VERBATIM from the scan.
- So L1 is gated on DATA ACCESS, not on method or engineering. One command away.

What holds up under grilling

- Pareto dominance; real-topology flagship (-28.4%, $p=1.7e-7$, wins 20/20).
- Beats the closest prior systems (CyGym-style, DAVA-style) head-to-head.
- Survived FIVE rounds of hostile self-review.
- Every headline number is backed by a committed result file and guarded by a test; experiments reproduce bit-for-bit; the paper compiles clean; 141 tests green.

PART 9

Outlook

Future work

- Validate on a real host-level enterprise inventory (L1) -- the highest-value next step.
- A jointly-adaptive adversary (payload + timing + placement together).
- End-to-end containment for polymorphic (multi-exploit) worms.
- A learned GNN policy; an LLM tool-use agent reading structured threat intel.
- An emulation bridge (packet-level) from the abstract model.

Defend what's spreading -- not just who's connected.

Infer the unseen exploit from the shape of the outbreak, and defend the machines that can actually catch it.